

gcongr

Pascal Herbrich

June 2026

This file was written at the state of the last commit on May 10, 2026. So it is outdated, however the changes are not too important to understand gcongr, since the main reason for the changes are for the grw tactic so far (status: June 17, 2026). Further, this file was planned for my first prepared talk, where I wanted to show meta programming, hence it has a little bit different contents.

1 Overview

An older version of the gcongr documentation states:

The 'gongr' tactic applies 'generalized congruence' rules, reducing a relational goal between a LHS and RHS.

On the first view, it seems easy. However, if you want to understand it in detail there appears several questions:

- Which rules are 'generalized congruence' rules? And how can we identify them?
- What is a relational goal?
- What should the LHS and RHS look like?

So to get a first impression, let's take a look at the example given in the docstrings.

```
example {a b x c d : ℝ} (h1 : a + 1 ≤ b + 1) (h2 : c + 2 ≤ d + 2) :  
  x ^ 2 * a + c ≤ x ^ 2 * b + d := by  
  -- LHS and RHS both have the form x ^ 2 * ?_ + ?_  
  gcongr x ^ 2 * ?_ + ?_  
  · -- New goal: ⊢ a ≤ b  
    linarith  
  · -- ⊢ New goal: c ≤ d  
    linarith
```

Notice that our relation is given by $\leq$, which we call a relation in mathematics, that the LHS and RHS have a similar structure, and that we can give gcongr a pattern to match for the generalized congruence rules. However, if we go a little bit in a detail for this, we get much more information about, how gcongr works. For this, we can write:

```
set_option trace.Meta.gcongr true
```

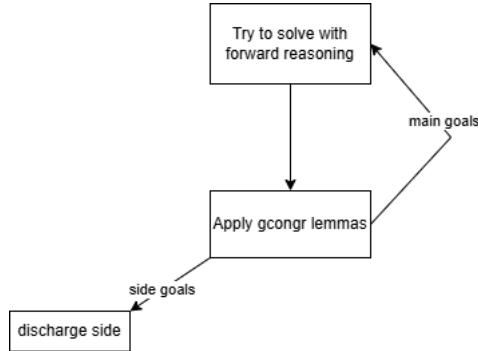
And now we can click through the trace points and see that there are trace points for `gcongr` and `gcongr_forward` and **Attempting to discharge side goal**. So it seems like `gcongr` distinguishes between some kind of different goals (side goals and main goals) and has a little bit of forward reasoning. There are three additional ways to configure the use of `gcongr`

1. `gcongr with x y ... z` allows you to name the implicit variables that `gcongr` introduces. This also works with `rintro` (recursive intro) patterns like `gcongr with <a, b> c`
2. `gcongr n`, where `n` is a natural number allows you to give a depth for the recursive use of `gcongr`
3. `gcongr t`, where `t` is a term, allows you to give `gcongr` a pattern to match for applying generalized congruence rules

Note that `gcongr` can be too aggressive, for example (this example is also taken from the docstring in the `gcongr` core file)

```
example {a b c d x : ℝ} (h : a + c + 1 ≤ b + d + 1) :
  x ^ 2 * (a + c) + 5 ≤ x ^ 2 * (b + d) + 5 := by
  -- Using a pattern to limit the depth.
  gcongr x ^ 2 * ?_ + 5 -- or 'gcongr 2'
  -- New goal: ⊢ a + c ≤ b + d
  linarith
```

If you only would apply `gcongr`, then all that's left for you to do is point out the wrong goals, that $a \leq b$ and $c \leq d$. Hence 2. and 3. are useful to tame `gcongr`. In the following we want to think of the `gcongr` cycle as in the diagram



2 Relations

As we have seen in the overview, $\leq$ is a relation for `gcongr`. But why? The `gcongr` algorithm uses the `getRel` function to extract a relation from a target, which is implemented as follows

```

def getRel (e : Expr) : Option (Name × Expr × Expr) :=
  match e with
  | .app (.app rel lhs) rhs => rel.getAppFn.constName?.map (·, lhs, rhs)
  | .forallE _ lhs rhs _ =>
    if !rhs.hasLooseBVars then
      some ('_Implies, lhs, rhs)
    else
      none
  | _ => none

```

It takes an expression as input and returns a tuple out of a name of a relation, and the LHS and the RHS as expressions again. Let's first look at the simple match, the first one. There we see that our target must have the following form:

$$\text{rel} \rightarrow \text{lhs} \rightarrow \text{rhs}$$

So we can represent this in the Lean model and define a simple relationship using

```

@[irreducible]
def r : ℕ → ℕ → Prop := fun _ ↦ fun _ ↦ sorry

```

The second kind of relations are implications. They joined `gcongr` about one year ago. So `gcongr` can now also handle tasks of the form

```

example (a b c d : ℕ) (h1 : r c a) (h2 : r b d) [IsTrans ℕ r] : r a b →
  r c d := by
  gcongr

```

3 Tagging lemmas and tactics

`Gcongr` takes the goal and attempts to apply the relevant lemmas automatically. The most of those lemmas will be lemmas that are tagged with `@[gcongr]`. Each Lean file has a `.olean` file which is not really readable for users but stores information for Lean. For example, there are the so called `EnvironmentExtensions`. In particular, there is an extension for `gcongr` and one for `gcongr_forward`. Every time you tag a statement with `@[gcongr]` or `@[gcongr_forward]`, it will be added to the `.olean` file. Lean then extracts the extensions from all imported files. For `gcongr`, the format of the extension is given by

```

/-- Structure recording the data for a "generalized congruence"
    ('gcongr') lemma. -/
structure GCongrLemma where
  /- The key under which the lemma is stored. -/
  key : GCongrKey
  /- The name of the lemma. -/
  declName : Name
  /- 'mainSubgoals' are the subgoals on which 'gcongr' will be
      recursively called. They store
  - the index of the hypothesis

```

```

- the number of parameters in the hypothesis
- whether it is contravariant (i.e. switches the order of the two
  arguments) -/
mainSubgoals : Array (Nat × Nat × Bool)
/-- The number of arguments that 'declName' takes when applying it. -/
numHyps : Nat
/-- The given priority of the lemma, for example as '@[gcongr high]'.
  -/
prio : Nat
/-- The number of arguments in the application of 'head' that are
  different.
This is used for sorting the lemmas.
For example, 'a + b ≤ a + c' has 'numVarying := 1'. -/
numVarying : Nat
deriving Inhabited

```

Note that Lean sorts the lemmas by “numVarying” to achieve the fastest runtime. The only exception for those lemmas is the following

```

variable {α : Sort*} {r : α → α → Prop} [IsTrans α r] {a b c d : α}

lemma rel_imp_rel (h₁ : r c a) (h₂ : r b d) : r a b → r c d :=
  fun h => IsTrans.trans c b d (IsTrans.trans c a b h₁ h) h₂

```

They claimed that it is too general to be tagged with @[gcongr]. Therefore, it is manually added to the list of lemmas to be considered in the core function of gcongr. The interesting part for now is, how we add a lemma in this style out of the given Expression. For this, we first analyse the function

```

def makeGCongrLemma (hyps : Array Expr) (target : Expr) (declName :
  Name) (prio : Nat) : MetaM GCongrLemma

```

It first checks, whether the goal is of the form $f x_1 \dots x_n \sim f y_1 \dots y_n$ for a relation $\sim$. The challenge, then, is to decide which hypothesis is a main goal and which one is only a side goal. For this, it first iterates over the parameter and collects all pairs of free variables. Then it applies `forallTelescopeReducing` and that works as follows: It takes an expression like $\forall x_1 \dots \forall x_n P(x_1, \dots, x_n)$ and introduces new free variables y_i locally (which are subsequently removed) for all x_i ($i \in \{1, \dots, n\}$), so that we can work with the expression $P(y_1, \dots, y_n)$ and don't have to take care about the $\forall$. Then `makeGCongrLemma` checks if $P(y_1, \dots, y_n)$ is of the form of a relation out of a pair of free variables in the goal. The relation doesn't have to be the same as in our goal. Since `gcongr` also works with implications, it could be, that the telescope has introduced a new free variable for the left-hand side of the implication. Hence, `makeGCongrLemma` also checks whether the last free variable and the goal together form a pair of free variables of the goal. So we get the cases, where we want to try another `gcongr`, and hypothesis like $x \geq 0$ are later side goals. So as cool as it looks like, the function `makeGCongrLemma` is only half of the magic. The other part is the 'add' parameter when we register our new attribute. First of all, we see that we have the option of adding a priority (in the form of a natural number with

a default value of 1000) to our new lemma and an attribute kind (global, local or scoped). Hence an example looks like

```
@[local gcongr 10]
```

The add function then applies a forallTelescope again, which covers some special cases and handles them separately. Then it tries to simply add it with

```
gcongrExt.add (← makeGCongrLemma xs type declName prio) kind
```

where `xs` are the hypotheses, `type` is the goal, `declName` is the name of the axiom/lemma/theorem/proposition, `prio` is the priority and `kind` is the attribute kind. If that fails, the function tests if the goal is a $\leftrightarrow$. Because normally a $\leftrightarrow$ can be viewed already as a relation, so for example

```
@[gcongr]
lemma my_iff {a b : ℕ} [Std.Symm r] : r a b ↔ r b a := by
  exact comm
```

would be added by the function above. But for a general $\leftrightarrow$ the left hand side and right hand side of the equivalence do not have to match a pattern, although they contain useful information for `gcongr`. Hence, the function tries to add one implication of the equivalence (viewed with the goal as the target of the implication and the condition is added to the hypotheses). Look at the example

```
@[gcongr]
theorem coe_lt_coe [LT α] {p : α → Prop} {x y : Subtype p} : (x : α) <
  y ↔ x < y :=
  Iff.rfl
```

This can't be added normally, because the parameter are not suitable (they must be pairs of definitionally equal variables or proof or must be otherwise free variables). But we still want to remove the coercion in the goal with `gcongr`. So the function adds an auxiliary lemma of the form:

```
example [LT α] {p : α → Prop} {x y : Subtype p} (h : x < y) : (x : α) <
  y := by
  gcongr
```

Notice that the other direction

```
example [LT α] {p : α → Prop} {x y : Subtype p} (h : (x : α) < y) : x <
  y := by
  gcongr
```

can't be added, because the goal is not of the form $f x_1 \dots x_n \sim f y_1 \dots y_n$. Notice the following important implementation detail: The function attempts to apply the implication from the right hand side to the left hand side and **if that fails** it tries to add the implication from the left to the right. So here

```
@[irreducible]
def f : ℕ → ℕ := fun _ => sorry
```

```
@[irreducible]
```

```

def g : ℕ → ℕ := fun _ ↦ sorry

@[gcongr]
lemma f_iff_g (a b : ℕ): f a < f b ↔ g a < g b := by sorry

example (a b : ℕ) (h: f a < f b) : g a < g b := by gcongr

example (a b : ℕ) (h: g a < g b) : f a < f b := by gcongr

```

does only the second example type check. In the end, the function tries to add a lemma as an implication for `gcongr` out of the hypotheses. For this, it iterates over the list of hypotheses and if it finds one with the same function, it removes it from the hypotheses, chains it with an $\forall$ in front of the goal, and tries to add this lemma (possibly under a new name, if it wasn't the last hypothesis, so that the order has changed). Notice here, that we only test the last hypothesis in the list with that function.

For the forward reasoning the extension has the form

```

/-- An extension for 'gcongr_forward'. -/
structure ForwardExt where
  eval (h : Expr) (goal : MVarId) : MetaM Unit

```

Remember that a goal is one of the two ways of considering a metavariable, so it has the type `MVarId`. Some important examples are given in the core file of `gcongr`:

```

/-- See if the term is 'a = b' and the goal is 'a ~ b' or 'b ~ a',
    with '~' reflexive. -/
@[gcongr_forward] def exactRefl : ForwardExt where
  eval h goal := do
    let m ← mkFreshExprMVar none
    goal.assignIfDefEq (← mkAppOptM 'Eq.subst #[h, m])
    goal.applyRfl

/-- See if the term is 'a ~ b' with '~' symmetric and the goal is 'b ~
    a'. -/
@[gcongr_forward] def symmExact : ForwardExt where
  eval h goal := do (← goal.applySymm).assignIfDefEq h

@[gcongr_forward] def exact : ForwardExt where
  eval e m := m.assignIfDefEq e

```

The 'add' function of the `gcongr_forward` extension is straightforward and does not involve any complex logic, but there are some technical details for which we will not go into further detail here. You simply get the environment make a `ForwardExt` out of the declaration name, and then add it in front of the other `ForwardExt` lemmas with the name and a tuple out of the name and the created `ForwardExt`.

4 Forward reasoning

For the default forward reasoning, the code is the following

```
def _root_.Lean.MVarId.gcongrForward (hs : Array Expr) (g : MVarId) :
  MetaM Bool :=
  withReducible do
    withTraceNode 'Meta.gcongr (fun _ => return m!"gcongr_forward: ⌊
      {← g.getType}" ) do
      -- Iterate over a list of terms
      let tacs := (forwardExt.getState (← getEnv)).2
      let mctx ← getMCtx
      for h in hs do
        try
          tacs.firstM fun (n, tac) =>
            withTraceNode 'Meta.gcongr (return m!"{.emoji} trying {n} on
              {h} : {← inferType h}" ) do
              tac.eval h g
              return true
          catch _ => setMCtx mctx
      return false
```

Personally, I don't like the double emoji, hence you can rewrite the second TraceNode as follows

```
withTraceNode 'Meta.gcongr (fun _ => return m!"trying {n} on {h} : {←
  inferType h}" ) do
```

This function takes a list of hypotheses and the goal and works with the reducible transparency settings. It gets all tactics from the imported files, then saves the current context and tries each tactic on each hypothesis. The main function, which gets all important hypotheses and then simply runs the function above, looks as follows:

```
def gcongrForwardDischarger (goal : MVarId) : MetaM Bool :=
  Elab.Term.TermElabM.run' do
    let mut hs := #[]
    -- collect the nontrivial hypotheses
    for h in ← getLCtx do
      if !h.isImplementationDetail then
        hs := hs.push (.fvar h.fvarId)
    -- run 'Lean.MVarId.gcongrForward' on each one
    goal.gcongrForward hs
```

5 Side goals

If the main function wants to solve side goals, it uses the following discharger in default:

```
def gcongrDischarger (goal : MVarId) : MetaM Unit :=
  Elab.Term.TermElabM.run' do
```

```

    trace[Meta.gcongr] "Attempting to discharge side goal {goal}"
    let [] ← Elab.Tactic.run goal <| Elab.Tactic.evalTactic
      (Unhygienic.run '(tactic| gcongr_discharger))
    | failure

```

If you want to get also an emoji, you can rewrite the trace line with

```

withTraceNode 'Meta.gcongr (fun _ => return m! "attempting to
  discharge side goal {goal}") do

```

So it only tries to execute the `gcongr_discharger` tactic and that tactic only tries to execute positivity and assumption. But you can modify it for your purposes if you want, and try out other strategies, for example:

```

macro_rules | '(tactic| gcongr_discharger) => '(tactic| your_tactic)

```

6 Main function

Gcongr has its own Monad Structure, given as follows:

```

structure State where
  /-- The new goals produced by 'gcongr'. This includes side-goals and
    main goals.
  TODO?: split this into separate side and main goals, so that the side
    goals can be
  discharged using a nested 'gcongr by ...' syntax. -/
  newGoals : Array MVarId := #[]
  /-- Patterns to use when doing intro. -/
  patterns : List (TSyntax 'rintroPat)

  /-- The context used by 'GCongrM'. -/
structure Context where
  /-- The discharger for main goals. -/
  mainGoalDischarger : MVarId → MetaM Bool
  /-- The discharger for side goals. -/
  sideGoalDischarger : MVarId → MetaM Unit

  /-- The monad used internally in the 'gcongr' tactic. -/
abbrev GCongrM := ReaderT Context <| StateRefT State MetaM

```

So we see, that this is implemented as a transformation of the State Monad for the states (consisting out of all new goals and the names for introducing new variables) and the Reader Monad for the context (consisting out of the two discharger). This makes sense, since we currently don't want to change our discharger in a computation, but have to add `newGoals`, which are those, that `gcongr` can't solve and survive until somebody else executes them; and after we have introduced some variables, we have to modify `patterns`. See for example in the four functions, that we use to interact with our new monad:

```

  /-- Add an unsolved goal to the 'newGoals' array in the state. -/
private def pushNewGoal (g : MVarId) : GCongrM Unit :=

```



```

    modify fun s ↦ { s with newGoals := s.newGoals.push g}

/-- Run the 'intro' tactic 'n' times on the given goal. -/
private def introN (goal : MVarId) (n : Nat) : GCongrM (List MVarId) :=
  do
    let (goals, patterns) ← goal.rintroWithPats (← get).patterns n
    modify ({· with patterns})
    return goals

/-- Run the discharger for main goals. -/
private def dischargeMain (mvarId : MVarId) : GCongrM Bool := do
  (← read).mainGoalDischarger mvarId

/-- Run the discharger for side goals. -/
private def dischargeSide (mvarId : MVarId) : GCongrM Unit := do
  let mctx ← getMCtx
  try (← read).sideGoalDischarger (← mvarId.intros).2
  catch _ => setMCtx mctx; pushNewGoal mvarId

```

We will later use the monad run function, given by

```

/-- Run a 'GCongrM' computation in 'MetaM'. -/
def GCongrM.run {α} (x : GCongrM α) (patterns : List (TSyntax
  'rintroPat) := [])
  (mainGoalDischarger : MVarId → MetaM Bool :=
    gcongrForwardDischarger)
  (sideGoalDischarger : MVarId → MetaM Unit := gcongrDischarger) :
  MetaM (α × Array MVarId) := do
  let (a, s) ← (x { mainGoalDischarger, sideGoalDischarger }).run {
    patterns }
  return (a, s.newGoals)

```

That is not particularly spectacular, we mention it here, because it is the spot where we set the `gcongrForwardDischarger` and `gcongrDischarger` which are explained above. Now we are ready to look at the core implementation of `gcongr`. The head is given by

```

partial def _root_.Lean.MVarId.gcongr
  (g : MVarId) (mdataLhs? : Option Bool)
  (depth : Nat := 1000000) :
  GCongrM Bool

```

so it takes the goal, a boolean, if we have matched the lhs with a pattern, and a depth as an upper bound to how often it applies `gcongr` and returns whether the function has made progress. First of all it checks whether there was no pattern given, then it tries to solve the goal with `rfl` or if it has the form $p \vdash p$ (namely `applyRflOrId`). If that fails, it also tries to run the `mainGoalDischarger`. If that also fails or there was a pattern, it decreases the depth or ends the calculation, if the depth is 0. Then it gets again the relation and lhs and rhs and joins the case, where a pattern was given. In that case it checks if the outer most expression has a hole and if so it tries the `mainGoalDischarger` or

adds the goal the the newGoals, if the discharger fails. After that checks, it checks, whether it contains a hole in a sub expression at all. If not, it again tries `applyRf10rId` or raises an error. After that, the function collects all suitable lemmas for that situation from the environment extension. If the relation is an implication, it additionally adds

```
variable {α : Sort*} {r : α → α → Prop} [IsTrans α r] {a b c d : α}
```

```
lemma rel_imp_rel (h1 : r c a) (h2 : r b d) : r a b → r c d :=
  fun h => IsTrans.trans c b d (IsTrans.trans c a b h1 h) h2
```

manually, because it is too general. Then it goes through all such lemmas and tries to apply them. I would again adjust the trace with an

```
withTraceNode 'Meta.gcongr
  (fun _ => return m!"trying {lem.declName}") do
```

right before it tries to apply a lemma. Because then you can check which lemmas are tried out, just like in the forward reasoning case. If a lemma succeeds, it gets all main subgoals of the lemma and tries to solve all side goals with the `sideGoalDischarger` and recursively apply itself on the main goals. In the end, if there was no template, and there was no lemma to apply it adds the goal to the `newGoals` and returns that it doesn't have make progress. If nothing worked, it fails with error for the missing cases.

Now let's analyse what the main function does, if somebody uses `gcongr`. It gets the main goal and sets up the classical important variables. Then it runs the run function of the monad on the core function above on the goal, and gets from it a boolean if it did progress and the remaining goals to solve. How it applies it is explained above, but after that, it checks whether it did progress and replaces the goal with the remaining goals or returns an error if no progress has been made. To prepare the application of the run function, it checks whether a template is given. If not it can run it without any restrictions. Otherwise if there is something, it checks whether it is a number, then it is just the depth, or otherwise it must be the pattern. Hence one can't use `gcongr` with a template for the lemmas and a depth. For every `?_` in the template it writes an annotation in front of it, such that it can works internally better with getting the holes. Then if there are holes and if it is definitionally equal to the lhs of the relation in the goal (or in other words, that the template makes sense), it instantiates meta variables (substitutes assigned meta variables from the context with their value) and replaces the lhs with it. So to be precise, the question above, whether there was a template is simplified and we would ask, whether there is meta data in the lhs. But with the identification of the lhs and the given meta data with the template, the question is the same, but could be misunderstood. The code looks as follows:

```
syntax (name := gcongr) "gcongr" (ppSpace colGt term)?
  (" with" (ppSpace colGt rintroPat)*)? : tactic
```

```

elab_rules : tactic
| `(tactic| gcongr $[template]? $[with $ps*]?) => do
  let g ← getMainGoal
  g.withContext do
    let type ← withReducible g.getType'
    let some (_rel, lhs, _rhs) := getRel type
    | throwError "gcongr failed, not a relation"
    -- The patterns from the 'with x y z' list
    let patterns := (ps?.getD #[]).toList
    -- Time to actually run the core tactic 'Lean.MVarId.gcongr'!
    let (progress, unsolvedGoals) ← do
      let some e := template | g.gcongr none |>.run patterns
      if let some depth := e.raw.isNatLit? then
        g.gcongr none depth |>.run patterns
      else
        -- Elaborate the template (e.g. 'x * ?_ + _')
        -- First, we replace occurrences of '?_' with 'gcongrHole% ?_'
        let e ← e.raw.replaceM fun stx =>
          if stx.isOfKind 'Parser.Term.syntheticHole then '(gcongrHole%
            _%$stx) else pure none
        let patt ← withTheReader Term.Context ({ · with
          ignoreTCFailures := true, errToSorry := false }) <|
          Term.elabTerm e (← inferType lhs)
        unless containsHoleAnnotation patt do
          throwError "invalid pattern {patt}, it doesn't contain any '?_'"
        unless ← withReducible <| isDefEq patt lhs do
          throwError "invalid pattern {patt}, it does not match with {lhs}"
        let patt ← instantiateMVars patt
        let g ← g.replaceTargetDefEq (updateRel type patt true)
        g.gcongr true |>.run patterns
    if progress then
      replaceMainGoal unsolvedGoals.toList
    else
      throwError "gcongr did not make progress"

```

7 Rel tactic

The rel tactic is a finishing tactic, that is similar to gcongr. You can write

```
rel [h1, h2, h3]
```

to solve a goal with hypotheses h1, h2 and h3. Internally the rel tactic runs a normal gcongr on the goal, but instead of the default gcongrForwardDischarger it builds a similar discharger out of the normal `_root_.Lean.MVarId.gcongrForward` function only with the given hypotheses. If gcongr can solve the goal and all side goals, rel succeeds, but if not it fails and returns all missing goals in the error message. The code is the following:

```
syntax "rel" " [" term,* "]" : tactic
```

```

elab_rules : tactic
| '(tactic| rel [$hyps,*]) => do
  let g ← getMainGoal
  g.withContext do
    let hyps ← hyps.getElems.mapM (elabTerm · none)
    let some (_rel, lhs, rhs) := getRel (← withReducible g.getType')
      | throwError "rel failed, goal not a relation"
    unless (← isDefEq (← inferType lhs) (← inferType rhs)) do
      throwError "rel failed, goal not a relation"
    -- The core tactic 'Lean.MVarId.gcongr' will be run with main-goal
    -- discharger being the tactic
    -- consisting of running 'Lean.MVarId.gcongrForward' (trying a term
    -- together with limited
    -- forward-reasoning on that term) on each of the listed terms.
    let assum g := g.gcongrForward hyps
    -- Time to actually run the core tactic 'Lean.MVarId.gcongr'!
    let (_, unsolvedGoals) ← g.gcongr none |>.run [] assum
    match unsolvedGoals.toList with
    -- if all goals are solved, succeed!
    | [] => pure ()
    -- if not, fail and report the unsolved goals
    | unsolvedGoals => do
      let unsolvedGoals ← liftMetaM <| List.mapM MVarId.getType
      unsolvedGoals
      let g := Lean.MessageData.joinSep (unsolvedGoals.map
      Lean.MessageData.ofExpr) Format.line
      throwError "rel failed, cannot prove goal by 'substituting' the
      listed relationships. \
      The steps which could not be automatically justified were:\n{g}"

```